Northeastern University

Assignment 2: EECE 7205

Professor: Dr. Fatema Nafa

Rohini Bidnurmath
NUID: 003916201

October 2026

Contents

1 Problem 1: Maximum Depth of a Binary Tree 3

2 Problem 2: Sum of All Nodes 5

3 Problem 3: Count Nodes with Exactly One Child 8

4 Problem 4: Are Two Trees Identical? 10

5 Problem 5: Minimum Value in a Binary Tree 12

6 Problem 6: Invert (Mirror) a Binary Tree 14

7 Problem 7: Print Each Level and Right Side View 16

8 Problem 8: Zigzag Level Order Traversal 19

9 Problem 9: Is the Tree Complete? 23

10 Problem 10: Lowest Common Ancestor 26

11 Problem 11: Build a Tree from Preorder and Inorder 29

12 Problem 12: Diameter of a Binary Tree 32

13 Problem 13: Serialize and Deserialize a Binary Tree 34

14 Problem 14: Is the Tree a Valid Binary Search Tree? 37

15 Problem 15: Vertical Order Traversal 40

Conventions Used in All Problems

- Reference tree (array $\{5, 12, 8, 21, 3, 17, 9\}$, $n = 7$): level 0: 5; level 1: 12, 8; level 2: 21, 3, 17, 9. Height $h = 2$, number of levels $L = h + 1 = 3$, and $n + 1 = 8$ NULL child pointers.
- Counting rules: every numbered line costs 1; a test counts every time it is evaluated; a recursive call counts once on the line that makes it (a line that makes two calls is counted once per execution); lines that only end a block are not counted; a **for** header is evaluated (iterations + 1) times.
- A recursive procedure on a tree with n nodes is called $n + (n + 1) = 2n + 1$ times: once per node and once per NULL pointer.
- In every "Worked Example" the calls are listed together with the lines they execute. The NULL calls under one node are shown in one row ("NULL children of x"), which lists the lines they execute (only the base-case lines).
- Every Worked Example starts with a drawing of the tree. Values in **bold red** are the nodes that matter for that example (the answer, the path, or the first difference).

1 Problem 1: Maximum Depth of a Binary Tree

1.1 Problem Statement

Return the number of edges on the longest path from the root to a leaf (an empty tree has depth -1). Input: reference tree. Expected output: 2.

1.2 Approach and Correctness

Recursive DFS. A NULL node has depth -1; any other node has depth $1 + \max(\text{left}, \text{right})$. By induction on subtree height, each call returns the correct depth of its subtree, so the root call returns the answer.

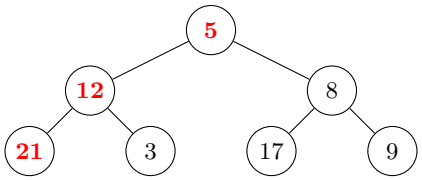
1.3 (a) Pseudocode

Input: *node* (root of a binary tree)
Output: integer, maximum depth in edges
Initial call: MaxDepth(root)
Procedure MaxDepth(*node*)

```
1: if node = NULL then
2:     return −1
3: L ← MaxDepth(node.left)
4: R ← MaxDepth(node.right)
5: return 1 + max(L, R)
```

1.4 (b) Worked Example

The tree (the longest path $5, 12, 21$ is marked):



Call	Node	Lines executed	Returns
1	5	1, 3, 4, 5	$1 + \max(1, 1) = 2$
2	12	1, 3, 4, 5	$1 + \max(0, 0) = 1$
3	21	1, 3, 4, 5	$1 + \max(-1, -1) = 0$
4, 5	NULL children of 21	1, 2	-1 each
6	3	1, 3, 4, 5	0
7, 8	NULL children of 3	1, 2	-1 each
9	8	1, 3, 4, 5	1
10	17	1, 3, 4, 5	0
11, 12	NULL children of 17	1, 2	-1 each
13	9	1, 3, 4, 5	0
14, 15	NULL children of 9	1, 2	-1 each

Final output: 2, which matches the expected output.

1.5 (c) Frequency Count

Line	Operation	Cost	Frequency	Count ($n = 7$)
1	test *node* = NULL	1	$2n + 1$	15
2	return -1	1	$n + 1$	8
3	call on left child	1	n	7
4	call on right child	1	n	7
5	compute and return	1	n	7
Total $T(n)$			$6n + 2$	44

Check: $6(7) + 2 = 44$, the same as the sum of the example column.

1.6 (d) Final Complexity

$T(n) = 6n + 2$, so the time is $\Theta(n)$ in the best, worst and average case, because every node must be visited. Extra space is $\Theta(h)$ for the recursion stack: $\Theta(\log n)$ for a balanced tree and $\Theta(n)$ for a chain, which is the worst case.

1.7 Twist: Single Chain of 7 Nodes

The frequency count does **not** change: it depends only on n, so $T(7) = 44$ for a chain as well (the same $2n + 1 = 15$ calls). What changes is the recursion depth: h grows from 2 to 6, so the stack space goes from $\Theta(\log n)$ to $\Theta(n)$.

1.8 Program Output

Please refer to the `output.txt` file in the GitHub link: https://github.com/roh9031/EECE---7205
Note on how the output file was created: A separate C++ file called `runner.cpp` was created to act as a wrapper around the main assignment file (`Assignment_2_FCE_Bidnurmath.cpp`) without altering its original code, allowing the program to run seamlessly. This was compiled using the PowerShell command:

```
g++ -std=c++17 runner.cpp -o assignment2_run.exe
```

The program was then executed, and its standard output was redirected directly into the output file using the PowerShell command:

```
.\assignment2_run.exe > output.txt
```

The resulting file contains the complete execution results for all 15 object-oriented binary tree algorithmic questions, standard test cases, and edge cases. For verification of all structural operations and traversals, refer to the `output.txt` file.

```
q1: max depth
2
```

2 Problem 2: Sum of All Nodes

2.1 Problem Statement

Return the sum of all node values. Input: reference tree. Expected output: 75. Implement it recursively and with a queue (level order), then compare.

2.2 Approach and Correctness

Recursive: $\text{sum}(node) = node.val + \text{sum}(left) + \text{sum}(right)$ with $\text{sum}(\text{NULL}) = 0$. *Queue:* every node is enqueued exactly once (its parent enqueues it) and its value is added when dequeued, so every value is counted exactly once.

2.3 (a) Pseudocode

Version 1: recursive

Input: *node* (root)
Output: sum of all values
Initial call: SumRec(root)
Procedure SumRec(*node*)

```
1: if node = NULL then
2:    return 0
3: return node.val + SumRec(node.left) + SumRec(node.right)
```

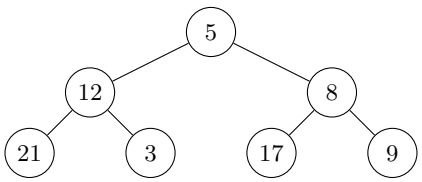
Version 2: queue

Input: *root*
Output: sum of all values
Initial call: SumQueue(root)
Procedure SumQueue(*root*)

```
1: if root = NULL then
2:    return 0
3: sum ← 0
4: Q ← empty queue
5: Enqueue(Q, root)
6: while Q is not empty do
7:    curr ← Dequeue(Q)
8:    sum ← sum + curr.val
9:    if curr.left ≠ NULL then
10:      Enqueue(Q, curr.left)
11:   if curr.right ≠ NULL then
12:      Enqueue(Q, curr.right)
13: end while
14: return sum
```

2.4 (b) Worked Example

The tree:



Recursive. Calls are listed in the order they return (post-order). Every node call executes lines 1 and 3; every NULL call executes lines 1 and 2 and returns 0.

Node	Lines	Returns
21 (leaf)	1, 3	$21 + 0 + 0 = 21$
3 (leaf)	1, 3	$3 + 0 + 0 = 3$
NULL children of 21 and 3 (4 calls)	1, 2	0 each
12	1, 3	$12 + 21 + 3 = 36$
17 (leaf)	1, 3	17
9 (leaf)	1, 3	9
NULL children of 17 and 9 (4 calls)	1, 2	0 each
8	1, 3	$8 + 17 + 9 = 34$
5 (root)	1, 3	$5 + 36 + 34 = 75$

Queue. Initially $Q = [5]$, $sum = 0$.

Pass	*curr*	*sum*	Queue after the pass
1	5	5	$[12, 8]$
2	12	17	$[8, 21, 3]$
3	8	25	$[21, 3, 17, 9]$
4	21	46	$[3, 17, 9]$
5	3	49	$[17, 9]$
6	17	66	$[9]$
7	9	75	$[\,]$

Both versions return 75, which matches the expected output.

2.5 (c) Frequency Count

Recursive version

Line	Operation	Cost	Frequency	Count ($n = 7$)
1	test $node =$ NULL	1	$2n + 1$	15
2	return 0	1	$n + 1$	8
3	add and make two calls	1	n	7
Total $T_{rec}(n)$			$4n + 2$	30

Queue version

Line	Operation	Cost	Frequency	Count ($n = 7$)
1	test $root =$ NULL	1	1	1
2	return 0	1	0	0
3	$sum \leftarrow 0$	1	1	1
4	create queue	1	1	1
5	enqueue root	1	1	1
6	while test	1	$n + 1$	8
7	dequeue	1	n	7
8	add value	1	n	7
9	test left child	1	n	7
10	enqueue left	1	$n - 1$ (shared with line 12)	3
11	test right child	1	n	7
12	enqueue right	1	(see line 10)	3
13	return *sum*	1	1	1
Total $T_{queue}(n)$			$6n + 5$	47

Here lines 10 and 12 together run $n-1$ times, because every node except the root is enqueued exactly once. In this tree the left enqueue runs 3 times and the right enqueue runs 3 times. Sum of the example column: $1+0+1+1+1+8+7+7+7+3+7+3+1 = 47 = 6(7)+5$. Check: the recursive total is $4(7)+2 = 30$.

2.6 (d) Final Complexity

Both versions are $\Theta(n)$ in the best, worst and average case, since every node is visited once ($4n+2$ vs. $6n+5$ steps). Extra space: recursive $\Theta(h)$ (worst case $\Theta(n)$ for a chain); queue $\Theta(w)$ where w is the maximum width (worst case $\Theta(n)$ for a perfect tree, where the last level holds about $n/2$ nodes; here the queue never holds more than 4 nodes).

2.7 Twist: Comparison

The recursive version has a smaller constant ($4n+2$ vs. $6n+5$) because it does no queue bookkeeping. The queue version avoids deep recursion, so it is safe for a chain; the recursive version is safe only when h is small. For a balanced tree the recursive stack ($\Theta(\log n)$) is smaller than the queue ($\Theta(n)$).

2.8 Program Output

Please refer to the `output.txt` file in the GitHub link: https://github.com/roh9031/EECE---7205

```
q2: sum of nodes
recursive: 75, queue: 75
```

3 Problem 3: Count Nodes with Exactly One Child

3.1 Problem Statement

Return the number of nodes that have a left child or a right child but not both. Input: the reference tree after INSERT(root, 4), so $n = 8$ and the new node 4 becomes the left child of 21 (level-order position 7). Expected output: 1.

3.2 Approach and Correctness

At each node, test whether exactly one child pointer is non-NULL; add 1 if so, plus the counts of both subtrees. Every node is examined once, so the total is exact. (In the code, INSERT is a real function: it scans in level order and attaches the new node to the first node with a free child slot, which is the left slot of 21.)

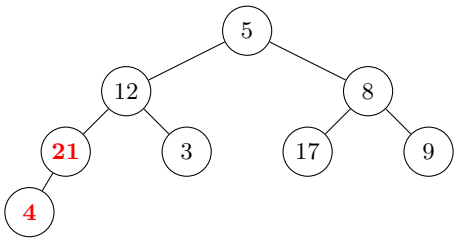
3.3 (a) Pseudocode

Input: *node* (root)
Output: number of nodes with exactly one child
Initial call: CountOne(root)
Procedure CountOne(*node*)

```
1: if node = NULL then
2:     return 0
3: c ← 0
4: if (node.left ≠ NULL ∧ node.right = NULL) ∨ (node.left = NULL ∧ node.right ≠ NULL) then
5:     c ← 1
6: return c + CountOne(node.left) + CountOne(node.right)
```

3.4 (b) Worked Example

Tree: 5; 12, 8; 21, 3, 17, 9; 4 (left child of 21). Only the node 21 has $c = 1$.



Every node call executes lines 1, 3, 4, 6 (and line 5 when it has exactly one child); every NULL call executes lines 1 and 2.

Node	Lines	c	Returns (line 6)
4, 3, 17, 9 (leaves)	1, 3, 4, 6	0	0 each
NULL children of 4, 3, 17, 9 (8 calls)	1, 2	—	0 each
21 (left = 4, right = NULL)	1, 3, 4, 5, 6	1	$1+0+0=1$
NULL right child of 21 (1 call)	1, 2	—	0
12 (two children)	1, 3, 4, 6	0	$0+1+0=1$
8 (two children)	1, 3, 4, 6	0	$0+0+0=0$
5 (root)	1, 3, 4, 6	0	$0+1+0=1$

That is $n + (n + 1) = 17$ calls (8 node calls and 9 NULL calls). Final output: 1, which matches the expected output.

3.5 (c) Frequency Count

Let k be the number of one-child nodes ($k = 1$ here, $n = 8$).

Line	Operation	Cost	Frequency	Count ($n = 8$)
1	test *node* = NULL	1	$2n + 1$	17
2	return 0	1	$n + 1$	9
3	$c \leftarrow 0$	1	n	8
4	one-child test	1	n	8
5	$c \leftarrow 1$	1	k	1
6	return with two calls	1	n	8
Total $T(n)$			$6n + 2 + k$	51

Check: $6(8) + 2 + 1 = 51$.

3.6 (d) Final Complexity

$T(n) = 6n + 2 + k$ with $0 \le k \le n - 1$, so the time is $\Theta(n)$ in the best and worst case. Extra space is $\Theta(h)$; the worst case is a chain ($h = n - 1$), where $\Theta(n)$ stack space is used.

3.7 Twist: Why Exactly One When n Is Even and None When n Is Odd

In a tree built from an array, node i has children at $2i + 1$ and $2i + 2$, whenever these are $< n$. Node i has exactly one child iff $2i + 1 = n - 1$, i.e. its left child is the last element and the right index $2i + 2 = n$ does not exist. Then $i = (n-2)/2$, which is an integer only when n is even. Every earlier node has $2i + 2 \le n - 1$ (two children) and every later node has $2i + 1 \ge n$ (none). So $k = 1$ for even n and $k = 0$ for odd n. Check: $n = 7 \Rightarrow k = 0$; after the insert $n = 8 \Rightarrow i = 3$, the node 21.

3.8 Program Output

Please refer to the `output.txt` file in the GitHub link: https://github.com/roh9031/EECE---7205

```
q3: one child count
1
```

4 Problem 4: Are Two Trees Identical?

4.1 Problem Statement

Given two roots, return true if the trees have the same shape and the same value at every position. Input: tree A and tree B from the array, and tree C from $\{5, 12, 8, 21, 3, 17, 10\}$. Expected output: $\textsc{Identical}(A, B) =$ true, $\textsc{Identical}(A, C) =$ false.

4.2 Approach and Correctness

Walk both trees in parallel. Two NULL nodes match; exactly one NULL means a shape difference; otherwise the values must be equal and both subtrees must match. The **and** is evaluated with short-circuiting, so the search stops at the first difference.

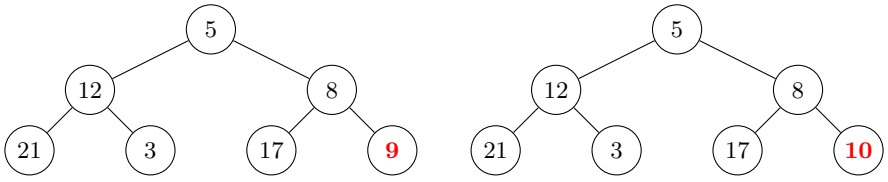
4.3 (a) Pseudocode

Input: p, q (roots of two trees)
Output: true or false
Initial call: $\textsc{Identical}(A,\ B)$
Procedure $\textsc{Identical}(p, q)$

1: **if** $p = \text{NULL} \wedge q = \text{NULL}$ **then**
2: **return** true
3: **if** $p = \text{NULL} \vee q = \text{NULL}$ **then**
4: **return** false
5: **return** $(p.val = q.val) \wedge \textsc{Identical}(p.left, q.left) \wedge \textsc{Identical}(p.right, q.right)$

4.4 (b) Worked Example

Tree A (= tree B) on the left and tree C on the right; the first difference, 9 vs. 10, is marked:



A vs. B (identical): 15 calls. The 7 node pairs execute lines 1, 3, 5 and return true; the 8 NULL pairs execute lines 1, 2 and return true. Result: true.

A vs. C (short-circuit stops at the first mismatch; 13 calls):

Call	(p, q)	Lines	Returns
1	$(5, 5)$	1, 3, 5	false (after call 13)
2	$(12, 12)$	1, 3, 5	true
3	$(21, 21)$	1, 3, 5	true
4, 5	(NULL, NULL) under 21	1, 2	true
6	$(3, 3)$	1, 3, 5	true
7, 8	(NULL, NULL) under 3	1, 2	true
9	$(8, 8)$	1, 3, 5	false (after call 13)
10	$(17, 17)$	1, 3, 5	true
11, 12	(NULL, NULL) under 17	1, 2	true
13	$(9, 10)$	1, 3, 5	false: $9 \neq 10$

The first difference is found at call 13: node 9 (right child of 8 in A) against node 10 in C. Result: false, as expected.

4.5 (c) Frequency Count

Line	Operation	Cost	Frequency (A vs. B)	Count A–B	Count A–C
1	test both NULL	1	$2n+1$	15	13
2	return true	1	$n+1$	8	6
3	test either NULL	1	n	7	7
4	return false	1	0 (same shape)	0	0
5	value test and calls	1	n	7	7
Total $T(n)$			$5n+2$	37	33

Check: $5(7)+2=37$. For A vs. C, the 6 NULL pairs are the children of 21, 3 and 17, so 13 calls are made instead of 15.

4.6 (d) Final Complexity

Best case $\Theta(1)$: the roots differ (or only one is empty), so one call decides. Worst case $\Theta(n)$: identical trees (or a difference at the last node visited), where all $2n+1$ calls are made. Extra space is $\Theta(h)$ for the recursion stack.

4.7 Twist

For IDENTICAL(A, C) the first difference appears at the pair $(9, 10)$, the right child of 8 (call 13). Best case: $\Theta(1)$ (different roots); worst case: $\Theta(n)$ (identical trees, or a mismatch at the last leaf).

4.8 Program Output

Please refer to the `output.txt` file in the GitHub link: https://github.com/roh9031/EECE---7205

```
q4: identical trees
identical(a, b) = true
identical(a, c) = false
```

5 Problem 5: Minimum Value in a Binary Tree

5.1 Problem Statement

Return the smallest value in the tree. The tree is not a BST, so every node must be checked. Input: reference tree. Expected output: 3.

5.2 Approach and Correctness

The minimum of a tree is the smallest of the root value, the minimum of the left subtree and the minimum of the right subtree. An empty subtree returns $+\infty$ (`INT_MAX` in the code), which never wins the comparison.

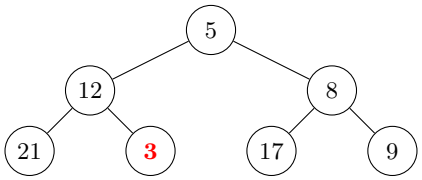
5.3 (a) Pseudocode

Input: *node* (root)
Output: smallest value
Initial call: MinValue(root)
Procedure MinValue(*node*)

1: **if** *node* = NULL **then**
2: **return** $+\infty$
3: $m \leftarrow node.val$
4: $m \leftarrow \min(m, \text{MinValue}(node.left))$
5: $m \leftarrow \min(m, \text{MinValue}(node.right))$
6: **return** m

5.4 (b) Worked Example

The tree (the minimum 3 is marked):



Each node call executes lines 1, 3, 4, 5, 6; each NULL call executes lines 1, 2 and returns $+\infty$.

Node	Lines	Value computed
21, 3, 17, 9 (leaves)	1, 3–6	$m = \min(v, +\infty, +\infty) = v$, so 21, 3, 17, 9
NULL children of the 4 leaves (8 calls)	1, 2	$+\infty$ each
12	1, 3–6	$m = 12 \to \min(12, 21) = 12 \to \min(12, 3) = 3$
8	1, 3–6	$m = 8 \to \min(8, 17) = 8 \to \min(8, 9) = 8$
5	1, 3–6	$m = 5 \to \min(5, 3) = 3 \to \min(3, 8) = 3$

Final output: 3, which matches the expected output.

5.5 (c) Frequency Count

Line	Operation	Cost	Frequency	Count ($n = 7$)
1	test $node = \text{NULL}$	1	$2n + 1$	15
2	return $+\infty$	1	$n + 1$	8
3	$m \leftarrow node.val$	1	n	7
4	min with left result	1	n	7
5	min with right result	1	n	7
6	return m	1	n	7
Total $T(n)$			$7n + 2$	51

Check: $7(7) + 2 = 51$.

5.6 (d) Final Complexity

Best, worst and average case are all $\Theta(n)$, since without ordering every node must be inspected. Extra space $\Theta(h)$; worst case $\Theta(n)$ for a chain.

5.7 Twist: If the Tree Were a BST

The minimum is the leftmost node: start at the root and follow *left* until $node.left = \text{NULL}$. The algorithm becomes a loop that visits at most $h + 1$ nodes, so $T = \Theta(h)$: $\Theta(\log n)$ for a balanced BST and $\Theta(n)$ for a chain-like BST (best case $\Theta(1)$ when the root has no left child). The iterative version uses $\Theta(1)$ extra space.

5.8 Program Output

Please refer to the `output.txt` file in the GitHub link: https://github.com/roh9031/EECE---7205

```
q5: minimum value
3
```

6 Problem 6: Invert (Mirror) a Binary Tree

6.1 Problem Statement

Swap the left and right child of every node and return the root. Input: reference tree. Expected output: level order after inverting is $5, 8, 12, 9, 17, 3, 21$.

6.2 Approach and Correctness

Swap the two child pointers of the current node using a temporary variable, then invert both subtrees. Every node is swapped exactly once, so the result is the mirror image.

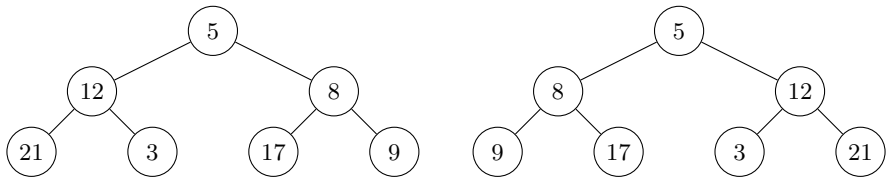
6.3 (a) Pseudocode

Input: *node* (root)
Output: root of the inverted tree
Initial call: INVERT(root)
Procedure INVERT(*node*)

1: **if** *node* = NULL **then**
2: **return** NULL
3: $temp \leftarrow node.left$
4: $node.left \leftarrow node.right$
5: $node.right \leftarrow temp$
6: INVERT($node.left$)
7: INVERT($node.right$)
8: **return** *node*

6.4 (b) Worked Example

Before (left) and after (right) inverting:



Calls are made in this order (after the swap, "left" means the new left child). The 8 NULL calls execute lines 1, 2.

Call	Node	Lines	Effect of lines 3–5
1	5	1, 3–8	children $(12, 8) \rightarrow (8, 12)$
2	8	1, 3–8	$(17, 9) \rightarrow (9, 17)$
3	9	1, 3–8	(NULL, NULL) swapped
4	17	1, 3–8	(NULL, NULL) swapped
5	12	1, 3–8	$(21, 3) \rightarrow (3, 21)$
6	3	1, 3–8	(NULL, NULL) swapped
7	21	1, 3–8	(NULL, NULL) swapped

The new tree is 5; $8, 12$; $9, 17, 3, 21$. Level order output: $5, 8, 12, 9, 17, 3, 21$, which matches the expected output.

6.5 (c) Frequency Count

Line	Operation	Cost	Frequency	Count ($n = 7$)
1	test $node =$ NULL	1	$2n + 1$	15
2	return NULL	1	$n + 1$	8
3	$temp \leftarrow node.left$	1	n	7
4	$node.left \leftarrow node.right$	1	n	7
5	$node.right \leftarrow temp$	1	n	7
6	recursive call (left)	1	n	7
7	recursive call (right)	1	n	7
8	return $node$	1	n	7
Total $T(n)$			$9n + 2$	65

Check: $9(7) + 2 = 65$.

6.6 (d) Final Complexity

$\Theta(n)$ in the best and worst case, since every node's pointers are swapped once. Extra space $\Theta(h)$; worst case $\Theta(n)$ for a chain.

6.7 Twist: Inorder Before and After

Before: 21 12 3 5 17 8 9. After: 9 8 17 5 3 12 21. The inorder after inverting is exactly the **reverse** of the inorder before. Inorder visits *left, node, right*; inverting swaps left and right at every node, so every node is visited as *right, node, left*, which is the reverse sequence.

6.8 Program Output

Please refer to the `output.txt` file in the GitHub link: https://github.com/roh9031/EECE---7205

```
q6: invert tree
5 8 12 9 17 3 21
```

7 Problem 7: Print Each Level and Right Side View

7.1 Problem Statement

Print the nodes of each level on its own line, then return the right side view (the last node of each level). Input: reference tree. Expected output: levels [5], [12, 8], [21, 3, 17, 9]; right side view 5, 8, 9.

7.2 Approach and Correctness

Level-order BFS. Before processing a level, record $size$ = current queue length; exactly those $size$ nodes are the whole level, and the children they enqueue form the next level. The last node ($i = size - 1$) of each level is the one visible from the right.

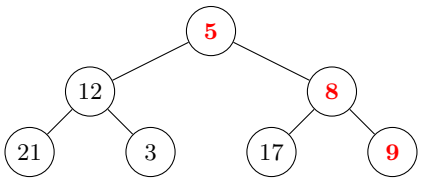
7.3 (a) Pseudocode

Input: *root*
Output: prints each level; returns list R (right side view)
Initial call: LevelAndRightView(root)
Procedure LevelAndRightView(*root*)

```
1:  if root = NULL then
2:    return empty list
3:  R ← empty list
4:  Q ← empty queue
5:  Enqueue(Q, root)
6:  while Q is not empty do
7:     size ← Size(Q)
8:     level ← empty list
9:     for i ← 0 to size − 1 do
10:       curr ← Dequeue(Q)
11:       append curr.val to level
12:       if i = size − 1 then
13:          append curr.val to R
14:       if curr.left ≠ NULL then
15:          Enqueue(Q, curr.left)
16:       if curr.right ≠ NULL then
17:          Enqueue(Q, curr.right)
18:    end for
19:    print level
20: end while
21: return R
```

7.4 (b) Worked Example

The tree (the right side view 5, 8, 9 is marked):



Pass (level)	Queue at start	*size*	Printed	R after
1	[5]	1	[5]	[5]
2	[12, 8]	2	[12, 8]	[5, 8]
3	[21, 3, 17, 9]	4	[21, 3, 17, 9]	[5, 8, 9]
end	[ ]			

Output: levels [5], [12, 8], [21, 3, 17, 9] and right side view 5, 8, 9, which matches the expected output.

7.5 (c) Frequency Count

Let $L = h + 1$ be the number of levels ($L = 3$).

Line	Operation	Cost	Frequency	Count ($n = 7$, $L = 3$)
1	test *root* = NULL	1	1	1
2	return empty list	1	0	0
3	$R \leftarrow \emptyset$	1	1	1
4	create queue	1	1	1
5	enqueue root	1	1	1
6	while test	1	$L + 1$	4
7	$size \leftarrow$ SIZE(Q)	1	L	3
8	$level \leftarrow \emptyset$	1	L	3
9	for-loop test	1	$n + L$	10
10	dequeue	1	n	7
11	append to *level*	1	n	7
12	test $i = size - 1$	1	n	7
13	append to R	1	L	3
14	test left child	1	n	7
15	enqueue left	1	$n - 1$ (with line 17)	3
16	test right child	1	n	7
17	enqueue right	1	(see line 15)	3
18	print *level*	1	L	3
19	return R	1	1	1
Total $T(n)$			$7n + 6L + 5$	72

Where each term comes from: the terms in n come from the per-node lines (9, 10, 11, 12, 14, 16, and lines 15 and 17 which together run $n - 1$ times); the terms in L come from the per-level lines (6, 7, 8, 9, 13, 18). Line 9 is evaluated $size + 1$ times in every level, giving $n + L$ in total. Check: $7(7) + 6(3) + 5 = 72 = 1 + 0 + 1 + 1 + 1 + 4 + 3 + 3 + 10 + 7 + 7 + 7 + 3 + 7 + 3 + 7 + 3 + 3 + 1$.

7.6 (d) Final Complexity

With $L = h + 1$, $T(n) = 7n + 6h + 11$, so the time is $\Theta(n)$ in every case (as $h \leq n - 1$). Extra space is $\Theta(w)$, the maximum width: $\Theta(1)$ for a chain (best case) and about $n/2$, i.e. $\Theta(n)$, for a perfect tree (worst case).

7.7 Program Output

Please refer to the `output.txt` file in the GitHub link: https://github.com/roh9031/EECE---7205

```
q7: levels & right view
levels:
[5]
[12, 8]
```

```
[21, 3, 17, 9]
right side view: 5, 8, 9
```

8 Problem 8: Zigzag Level Order Traversal

8.1 Problem Statement

Visit level 0 left to right, level 1 right to left, level 2 left to right, and so on. Input: reference tree. Expected output: $5, 8, 12, 21, 3, 17, 9$.

8.2 Approach and Correctness

Standard BFS with a queue. Each level is first stored in an array of size $size$; the node at position i of the level goes to index i (left to right) or $size - 1 - i$ (right to left), and the direction flag is toggled after each level. The array is appended to the result.

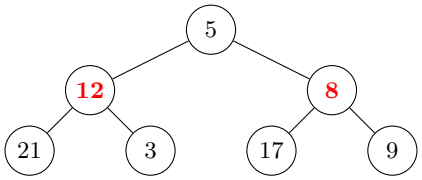
8.3 (a) Pseudocode

Input: *root*
Output: list of values in zigzag order
Initial call: ZIGZAG(root)
Procedure ZIGZAG(*root*)

```
result ← empty list
if root = NULL then
    return result
Q ← empty queue
ENQUEUE(Q, root)
leftToRight ← true
while Q is not empty do
    size ← SIZE(Q)
    level ← array of length size
    for i ← 0 to size − 1 do
        curr ← DEQUEUE(Q)
        index ← (leftToRight ? i : size − 1 − i)
        level[index] ← curr.val
        if curr.left ≠ NULL then
            ENQUEUE(Q, curr.left)
        if curr.right ≠ NULL then
            ENQUEUE(Q, curr.right)
    end for
    leftToRight ← ¬leftToRight
    for j ← 0 to size − 1 do
        append level[j] to result
    end for
end while
return result
```

8.4 (b) Worked Example

The tree (level 1 is read right to left, so 8 comes before 12):



Pass	Queue at start	Direction	*level* array	*result* after
1	[5]	L→R	[5]	5
2	[12, 8]	R→L (12 → index 1, 8 → index 0)	[8, 12]	5, 8, 12
3	[21, 3, 17, 9]	L→R	[21, 3, 17, 9]	5, 8, 12, 21, 3, 17, 9

Final output: $5, 8, 12, 21, 3, 17, 9$, which matches the expected output.

8.5 (c) Frequency Count

Let $L = 3$ be the number of levels.

Line	Operation	Cost	Frequency	Count ($n = 7$, $L = 3$)
1	$result \leftarrow \emptyset$	1	1	1
2	test $root =$ NULL	1	1	1
3	return *result*	1	0	0
4	create queue	1	1	1
5	enqueue root	1	1	1
6	$leftToRight \leftarrow$ true	1	1	1
7	while test	1	$L + 1$	4
8	$size \leftarrow$ SIZE(Q)	1	L	3
9	create *level*	1	L	3
10	for-loop test (fill)	1	$n + L$	10
11	dequeue	1	n	7
12	compute *index*	1	n	7
13	$level[index] \leftarrow curr.val$	1	n	7
14	test left child	1	n	7
15	enqueue left	1	$n - 1$ (with line 17)	3
16	test right child	1	n	7
17	enqueue right	1	(see line 15)	3
18	toggle direction	1	L	3
19	for-loop test (copy)	1	$n + L$	10
20	append to *result*	1	n	7
21	return *result*	1	1	1
Total $T(n)$			$9n + 6L + 6$	87

Check: $9(7) + 6(3) + 6 = 87$, equal to the sum of the example column.

8.6 (d) Final Complexity

$\Theta(n)$ in the best and worst case: every node is enqueued, dequeued, indexed and copied once. Extra space $\Theta(w)$ for the queue and the *level* array: $\Theta(1)$ for a chain, $\Theta(n)$ for a perfect tree (worst case).

8.7 Twist: Two Stacks or Queue + Reverse?

The question asks which of the two alternatives needs less extra work, so both are counted here with the same rules, on the same tree ($n = 7$, $L = 3$), and compared with the index-filling version above.

Alternative 1: queue + reverse every second level. Same as the pseudocode above, except that each level is filled in order (append), and a level that must go right to left is reversed before it is copied. Let $r = \lfloor L/2 \rfloor$ be the number of reversed levels and $s = \sum \lfloor size/2 \rfloor$ over those levels the number of swaps (here $r = 1$, $s = 1$ for the level $[12, 8]$).

Line	Operation	Cost	Frequency	Count
1–6	setup (result, test root, queue, enqueue, flag; line 3 does not run)	1	5 steps	5
7	while test	1	$L+1$	4
8, 9	*size* and empty *level*	1	$2L$	6
10	for-loop test (fill)	1	$n+L$	10
11, 12	dequeue, append to *level*	1	$2n$	14
13, 15	test left, test right	1	$2n$	14
14, 16	enqueue left and right	1	$n-1$	6
17	test "right to left?"	1	L	3
18	call REVERSE(*level*)	1	r	1
19	swap steps inside REVERSE	1	s	1
20	toggle direction	1	L	3
21	for-loop test (copy)	1	$n+L$	10
22	append to *result*	1	n	7
23	return *result*	1	1	1
Total			$8n+7L+6+r+s$	85

Check: $8(7)+7(3)+6+1+1=85$.

Alternative 2: two stacks. Stack *cur* holds the level being read, stack *next* collects the following level. Each node is popped from *cur* and appended to the result; its children are pushed on *next* (left then right on a left-to-right level, right then left on a right-to-left level, which reverses the order automatically). After a level, *cur* and *next* are swapped.

Line	Operation	Cost	Frequency	Count
1–6	setup (result, test root, two stacks, push root, flag)	1	5 steps	5
7	outer while test	1	$L+1$	4
8	inner while test (*cur* not empty)	1	$n+L$	10
9, 10	pop, append to *result*	1	$2n$	14
11	test direction (decides push order)	1	n	7
12, 14	test first child, test second child	1	$2n$	14
13, 15	push first child, push second child	1	$n-1$	6
16	swap *cur* and *next*	1	L	3
17	toggle direction	1	L	3
18	return *result*	1	1	1
Total			$7n+4L+6$	67

Check: $7(7)+4(3)+6=67$.

Comparison.

Version	$T(n)$	Count ($n=7$)	Extra space
Queue + index filling (used in the code)	$9n+6L+6$	87	queue + *level* array, $\Theta(w)$
Queue + reverse	$8n+7L+6+r+s$	85	queue + *level* array, $\Theta(w)$
Two stacks	$7n+4L+6$	67	two stacks, $\Theta(w)$

In the frequency count the **two-stack version does the least work** ($7n+4L+6$): it never builds a separate *level* array, so it has no index computation, no reversal and no copy loop, and the reversed order comes for free from the push order. Queue + reverse is a little cheaper than index filling in this model because it avoids the index computation (one step per node), but it adds one test per level and the swap steps. All three are $\Theta(n)$; the difference is only in the constant. The code keeps the index-filling version because it uses a single container and writes each value directly to its final position.

8.8 Program Output

Please refer to the `output.txt` file in the GitHub link: https://github.com/roh9031/EECE---7205

```
q8: zigzag traversal
5 8 12 21 3 17 9
```

9 Problem 9: Is the Tree Complete?

9.1 Problem Statement

Return true if every level is full except possibly the last and the last level is filled from the left. Input: tree P from the array, and tree Q = $\{5, 12, 8, \text{NULL}, 3, 17, 9\}$ (node 12 has no left child). Expected output: COMPLETE(P) = true, COMPLETE(Q) = false.

9.2 Approach and Correctness

Level-order traversal that also enqueues NULL children. In a complete tree, all real nodes come before the first NULL in level order. So once a NULL is dequeued, any later real node proves the tree incomplete.

9.3 (a) Pseudocode

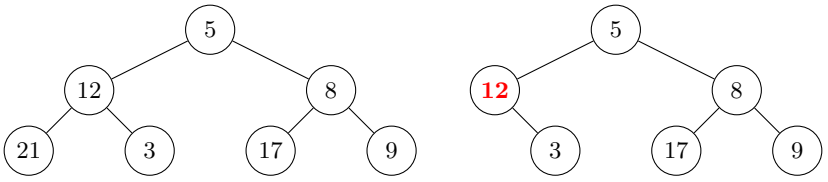
Input: *root*
Output: true or false
Initial call: ISCOMPLETE(root)
Procedure ISCOMPLETE(*root*)

```
1:  if root = NULL then
2:      return true
3:  Q ← empty queue
4:  ENQUEUE(Q, root)
5:  nullSeen ← false
6:  while Q is not empty do
7:      curr ← DEQUEUE(Q)
8:      if curr = NULL then
9:          nullSeen ← true
10:         continue with the next iteration
11:     if nullSeen then
12:         return false
13:     ENQUEUE(Q, curr.left)
14:     ENQUEUE(Q, curr.right)
15: end while
16: return true
```

(The code uses `if/else` for lines 8–14; the pseudocode above is equivalent and is the version whose lines are counted below.)

9.4 (b) Worked Example

Tree P (left) and tree Q (right; the gap is the missing left child of 12):



Tree P (items in the queue; N = NULL):

Pass	*curr*	Queue after the pass	*nullSeen*
1	5	$[12, 8]$	false
2	12	$[8, 21, 3]$	false
3	8	$[21, 3, 17, 9]$	false
4	21	$[3, 17, 9, N, N]$	false
5	3	$[17, 9, N, N, N, N]$	false
6	17	$[9, N \times 6]$	false
7	9	$[N \times 8]$	false
8–15	N (eight times)	[ ]	true, but no real node follows

The queue empties and the procedure returns true. Result: COMPLETE(P) = true.

Tree Q (node 12 has $left =$ NULL, $right = 3$):

Pass	*curr*	Queue after the pass	*nullSeen*
1	5	$[12, 8]$	false
2	12	$[8, N, 3]$	false
3	8	$[N, 3, 17, 9]$	false
4	N	$[3, 17, 9]$	true
5	3	—	line 11 is true, so **return false**

Result: COMPLETE(Q) = false, as expected.

9.5 (c) Frequency Count (tree P, $n = 7$)

The queue handles $2n + 1 = 15$ items (n real nodes and $n + 1$ NULL markers).

Line	Operation	Cost	Frequency	Count ($n = 7$)
1	test $root =$ NULL	1	1	1
2	return true	1	0	0
3	create queue	1	1	1
4	enqueue root	1	1	1
5	$nullSeen \leftarrow$ false	1	1	1
6	while test	1	$2n + 2$	16
7	dequeue	1	$2n + 1$	15
8	test $curr =$ NULL	1	$2n + 1$	15
9	$nullSeen \leftarrow$ true	1	$n + 1$	8
10	continue	1	$n + 1$	8
11	test *nullSeen*	1	n	7
12	return false	1	0	0
13	enqueue left	1	n	7
14	enqueue right	1	n	7
15	return true	1	1	1
Total $T(n)$			$11n + 11$	88

Check: $11(7) + 11 = 88$. For tree Q the loop stops after 5 passes.

9.6 (d) Final Complexity

Best case $\Theta(1)$: a gap is found early (e.g. a root with only a right child is rejected after 3 dequeues; tree Q after 5). Worst case $\Theta(n)$: a complete tree is processed completely ($11n + 11$ steps). Extra space $\Theta(w)$ for the queue, which includes NULL markers.

9.7 Twist: Why Insert and Delete Keep the Tree Complete

INSERT puts the new node at the first free position in level order, i.e. index n, directly after the last used slot, so the used positions $0, \ldots, n$ stay contiguous: no gap, and the last level is still filled from the left. DELETE copies the last node in level order (index $n-1$) into the deleted node's place and removes that last slot, so positions $0, \ldots, n-2$ remain contiguous. A tree whose nodes occupy exactly the positions $0, \ldots, n-1$ of the array is complete.

9.8 Program Output

Please refer to the `output.txt` file in the GitHub link: https://github.com/roh9031/EECE---7205

```
q9: complete tree
complete(p) = true
complete(q) = false
```

10 Problem 10: Lowest Common Ancestor

10.1 Problem Statement

Given two values a and b in the tree, return their lowest common ancestor (a node counts as being in its own subtree). Input: reference tree. Expected output: $\mathrm{LCA}(21, 3) = 12$; $\mathrm{LCA}(21, 9) = 5$; $\mathrm{LCA}(17, 8) = 8$.

10.2 Approach and Correctness

Single recursive pass. A call returns the node itself if it is NULL or equals a or b. If both subtrees return a non-NULL result, then a and b lie in different subtrees and the current node is the LCA; otherwise the non-NULL result is passed up. (This assumes both values exist in the tree.)

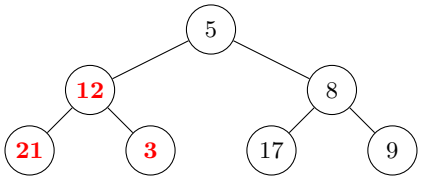
10.3 (a) Pseudocode

Input: *node*, values a, b
Output: LCA node
Initial call: LCA(root, a, b)
Procedure $\mathrm{LCA}(node, a, b)$

1: **if** $node = \text{NULL} \vee node.val = a \vee node.val = b$ **then**
2: **return** *node*
3: $left \leftarrow \mathrm{LCA}(node.left, a, b)$
4: $right \leftarrow \mathrm{LCA}(node.right, a, b)$
5: **if** $left \neq \text{NULL} \wedge right \neq \text{NULL}$ **then**
6: **return** *node*
7: **if** $left \neq \text{NULL}$ **then**
8: **return** $left$
9: **return** $right$

10.4 (b) Worked Example

The tree for $\mathrm{LCA}(21, 3)$ (the targets 21 and 3 and their ancestor 12 are marked):



LCA$(21, 3)$ (11 calls: the subtrees below 21 and 3 are never entered):

Call	Node	Lines executed	Returns
1	5	1, 3, 4, 5, 7, 8	12 (left found, right NULL)
2	12	1, 3, 4, 5, 6	12 (both sides found)
3	21	1, 2	21 (match)
4	3	1, 2	3 (match)
5	8	1, 3, 4, 5, 7, 9	NULL
6	17	1, 3, 4, 5, 7, 9	NULL
7, 8	NULL children of 17	1, 2	NULL
9	9	1, 3, 4, 5, 7, 9	NULL
10, 11	NULL children of 9	1, 2	NULL

LCA$(21, 9)$**:** 21 is found under 12 (returns 21 up), 9 is found under 8 (returns 9 up); at the root both are non-NULL, so the answer is 5. **LCA**$(17, 8)$**:** the root's right call reaches 8, which matches immediately and

returns 8 (never looking at 17); the left side returns NULL, so the answer is 8. All three results match the expected output.

10.5 (c) Frequency Count

The left column is the worst case (full traversal, $2n + 1$ calls, e.g. a value that is absent); the right column is the actual count for LCA$(21, 3)$.

Line	Operation	Cost	Frequency (full traversal)	Count LCA$(21, 3)$
1	combined test	1	$2n + 1$	11
2	return *node*	1	$n + 1$	6
3	call on left	1	n	5
4	call on right	1	n	5
5	test both non-NULL	1	n	5
6	return *node*	1	0	1
7	test $left \neq$ NULL	1	n	4
8	return *left*	1	0	1
9	return *right*	1	n	3
Total $T(n)$			$8n + 2$ (worst case)	41

Check: the worst-case formula gives $8(7)+2 = 58$ for a full traversal; the example finishes in 41 steps because 4 calls are skipped (its sum is $11 + 6 + 5 + 5 + 5 + 1 + 4 + 1 + 3 = 41$).

10.6 (d) Final Complexity

Best case $\Theta(1)$ (the root equals a or b); worst case $\Theta(n)$ (the targets are deep or the search must visit the whole tree). Extra space $\Theta(h)$.

10.7 Twist: FindPath vs. Single Pass

Method 1 finds the root-to-node path of a and of b with FINDPATH and then compares the two paths. (In the code this is `findPath` and `lcaByPaths`; it returns the same three answers.)

Input: *node*, value *target*, list *path*
Output: true if found; *path* then holds the root-to-target values
Initial call: FINDPATH(root, a, *pathA*)
Procedure FINDPATH($node, target, path$)

```
1: if node = NULL then
2:    return false
3: append node.val to path
4: if node.val = target then
5:    return true
6: if FINDPATH(node.left, target, path) or FINDPATH(node.right, target, path) then
7:    return true
8: remove the last value of path
9: return false
```

Input: *root*, values a, b
Output: LCA value
Initial call: LCAPATHS(root, a, b)
Procedure LCAPATHS($root, a, b$)

```
1: pathA ← empty list; pathB ← empty list
2: FINDPATH(root, a, pathA)
3: FINDPATH(root, b, pathB)
4: i ← 0
```

5: **while** $i < |pathA| \wedge i < |pathB| \wedge pathA[i] = pathB[i]$ **do**
6: $\quad i \leftarrow i + 1$
7: **end while**
8: **return** $pathA[i-1]$

Frequency count of FindPath (a line with two calls is counted once per execution). Each NULL call executes lines 1, 2; a node call that does not match and fails below executes lines 1, 3, 4, 6, 8, 9 (6 steps); a node call that finds the target below executes 1, 3, 4, 6, 7 (5 steps); the matching node executes 1, 3, 4, 5 (4 steps).

Search	Calls	Steps (example)	Steps (target absent, full traversal)
FINDPATH(root, 21)	$5, 12, 21$ (3 calls)	$5+5+4=14$	$6n+2(n+1)=8n+2$
FINDPATH(root, 3)	$5, 12, 21,$ NULL, NULL, 3 (6 calls)	$5+5+6+2+2+4=24$	$8n+2$

Frequency count of LCAPaths for LCA$(21, 3)$**:** paths $[5, 12, 21]$ and $[5, 12, 3]$. Line 1: 1; line 2: 1; line 3: 1; line 4: 1; line 5: evaluated 3 times ($i = 0, 1$ true, $i = 2$ false because $21 \neq 3$): 3; line 6: 2; line 7: 1. That is 10 steps in LCAPATHS itself plus $14 + 24 = 38$ inside the two searches, so $T_{method\,1} = 48$ (the two call lines 2 and 3 are included in the 10). The loop runs at most $h + 1$ times, so in general

$$T_{method\,1} \leq 2(8n + 2) + O(h) = 16n + O(h).$$

Method 2 (the single pass above) costs 41 steps for LCA$(21, 3)$ and at most $8n + 2$ in general.

Method	Example LCA$(21, 3)$	Worst case	Extra space
1: two FINDPATH + compare	48	$16n + O(h)$	$\Theta(h)$: stack and **both paths**
2: single pass	41	$8n + 2$	$\Theta(h)$: stack only

Both are $\Theta(n)$, but the single pass is cheaper: on this example 41 against 48 steps, and in the worst case about half the work ($8n + 2$ against $16n + O(h)$) because it traverses the tree once instead of twice, and it stores no paths.

10.8 Program Output

Please refer to the `output.txt` file in the GitHub link: https://github.com/roh9031/EECE---7205

```
q10: lowest common ancestor
lca(21, 3) = 12
lca(21, 9) = 5
lca(17, 8) = 8
by paths: 12, 5, 8
```

11 Problem 11: Build a Tree from Preorder and Inorder

11.1 Problem Statement

Rebuild the tree from preorder $5, 12, 21, 3, 8, 17, 9$ and inorder $21, 12, 3, 5, 17, 8, 9$ (distinct values). Expected output: the tree from the array (level order $5, 12, 8, 21, 3, 17, 9$; postorder $21, 3, 12, 17, 9, 8, 5$).

11.2 Approach and Correctness

The first preorder value is the root. Its position k in the inorder list splits the values into a left part (inorder $[inStart, k-1]$) and a right part. The left subtree has $numsLeft = k - inStart$ nodes, which are the next $numsLeft$ preorder values. A hash map from value to inorder index makes finding k an $O(1)$ operation.

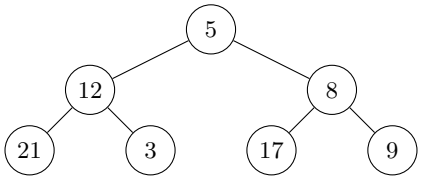
11.3 (a) Pseudocode

Input: preorder array pre, inorder array in (n distinct values)
Output: root of the rebuilt tree
Initial call: BUILDTREE(pre, in)
Procedure BUILDTREE(pre, in)

1: **for** $i \leftarrow 0$ **to** $n-1$ **do**
2: $\quad inMap[in[i]] \leftarrow i$
3: **end for**
4: **return** HELPER$(0, n-1, 0, n-1)$
Procedure HELPER$(preS, preE, inS, inE)$ (uses pre, in, $inMap$)
5: **if** $preS > preE \vee inS > inE$ **then**
6: $\quad$ **return** NULL
7: $root \leftarrow$ new node$(pre[preS])$
8: $k \leftarrow inMap[root.val]$
9: $numsLeft \leftarrow k - inS$
10: $root.left \leftarrow$ HELPER$(preS+1,\ preS+numsLeft,\ inS,\ k-1)$
11: $root.right \leftarrow$ HELPER$(preS+numsLeft+1,\ preE,\ k+1,\ inE)$
12: **return** $root$

11.4 (b) Worked Example

The rebuilt tree:



Map: $21 \to 0,\ 12 \to 1,\ 3 \to 2,\ 5 \to 3,\ 17 \to 4,\ 8 \to 5,\ 9 \to 6$ (lines 1–2 run 7 times, line 3 once). Each node call executes lines 4, 6, 7, 8, 9, 10, 11; each of the 8 calls with an empty range executes lines 4, 5 and returns NULL.

Call	$(preS, preE, inS, inE)$	Lines	$root$, k, $numsLeft$	Returns
1	$(0, 6, 0, 6)$	4, 6–11	5, 3, 3	node 5
2	$(1, 3, 0, 2)$	4, 6–11	12, 1, 1	node 12, left of 5
3	$(2, 2, 0, 0)$	4, 6–11	21, 0, 0	leaf 21, left of 12
4, 5	empty ranges under 21	4, 5	—	NULL each
6	$(3, 3, 2, 2)$	4, 6–11	3, 2, 0	leaf 3, right of 12
7, 8	empty ranges under 3	4, 5	—	NULL each
9	$(4, 6, 4, 6)$	4, 6–11	8, 5, 1	node 8, right of 5
10	$(5, 5, 4, 4)$	4, 6–11	17, 4, 0	leaf 17, left of 8
11, 12	empty ranges under 17	4, 5	—	NULL each
13	$(6, 6, 6, 6)$	4, 6–11	9, 6, 0	leaf 9, right of 8
14, 15	empty ranges under 9	4, 5	—	NULL each

The rebuilt tree has root 5, level order $5, 12, 8, 21, 3, 17, 9$ and postorder $21, 3, 12, 17, 9, 8, 5$, which matches the expected output (the program prints both).

11.5 (c) Frequency Count

Line	Operation	Cost	Frequency	Count ($n = 7$)
1	for-loop test	1	$n + 1$	8
2	$inMap[in[i]] \leftarrow i$	1	n	7
3	call HELPER	1	1	1
4	test empty range	1	$2n + 1$	15
5	return NULL	1	$n + 1$	8
6	create node	1	n	7
7	map lookup	1	n	7
8	$numsLeft$	1	n	7
9	left call	1	n	7
10	right call	1	n	7
11	return $root$	1	n	7
Total $T(n)$			$11n + 4$	81

Check: $11(7) + 4 = 81$.

11.6 (d) Final Complexity

$\Theta(n)$ in the best and worst case: every node is created once and the map lookups are $O(1)$. No input changes the step count ($11n + 4$ always); only the recursion depth varies, and the worst case is a skewed tree ($h = n - 1$). Extra space $\Theta(n)$ for the hash map plus $\Theta(h)$ recursion stack (the output tree itself is also $\Theta(n)$).

11.7 Twist: Without the Lookup Table

Without $inMap$, lines 1–2 disappear and line 7 becomes a linear search of $in[inS..inE]$ for the root. A call whose range has m elements and whose root sits at offset j (counting from 0) costs $j + 1 \leq m$ comparisons; call this cost S_{call} and let $S = \sum S_{call}$. The other lines are unchanged: line 3: 1; line 4: $2n+1$; line 5: $n+1$; lines 6, 8, 9, 10, 11: n each. So

$$T_{search}(n) = 1 + (2n + 1) + (n + 1) + 5n + S = 8n + 3 + S.$$

Worst case. In a skewed tree the root is the last element of its range every time, so the ranges shrink by one: $S = n + (n-1) + \cdots + 1 = n(n+1)/2$, and

$$T^{worst}_{search}(n) = 8n + 3 + \frac{n(n+1)}{2} = \frac{n^2 + 17n + 6}{2} = \Theta(n^2).$$

Best case. If the root is always the first element of its range ($S = n$), $T = 9n + 3 = \Theta(n)$.

On the example the comparisons per call are $4, 2, 1, 1, 2, 1, 1$, so $S = 12$ and $T_{search} = 8(7)+3+12 = 71$, which is actually smaller than the 81 of the table version: for $n = 7$ the table costs $2n + 1 = 15$ extra steps to build. The table pays off as n grows, because S can grow like $n^2/2$ while the table version is $11n + 4$ for every input.

With the lookup table, building it costs $\Theta(n)$ once (lines 1–2) and every lookup is $\Theta(1)$, so the total is $11n + 4 = \Theta(n)$ for every input, instead of $\Theta(n^2)$ in the worst case.

11.8 Program Output

Please refer to the `output.txt` file in the GitHub link: https://github.com/roh9031/EECE---7205

```
q11: build from preorder & inorder
rebuilt root value: 5
level order: 5 12 8 21 3 17 9
postorder: 21 3 12 17 9 8 5
```

12 Problem 12: Diameter of a Binary Tree

12.1 Problem Statement

Return the number of edges on the longest path between any two nodes (the path need not pass through the root). Input: reference tree. Expected output: 4 (e.g. path $21, 12, 5, 8, 17$).

12.2 Approach and Correctness

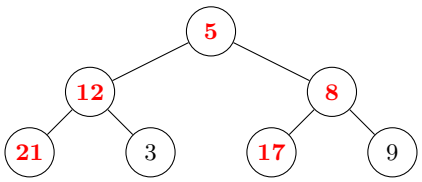
Post-order: each call returns the height of its subtree and, using the heights of its two children, updates the best path through the current node, $leftH + rightH + 2$ edges. The diameter is the maximum over all nodes, because the longest path has a highest node and passes through it.

12.3 (a) Pseudocode

Input: *root*
Output: diameter in edges
Initial call: DIAMETER(root)
Procedure DIAMETER(*root*)
1: $d \leftarrow 0$
2: HEIGHTDIAM(*root*, d)
3: **return** d
Procedure HEIGHTDIAM(*node*, d) (d is passed by reference)
4: **if** *node* = NULL **then**
5: **return** -1
6: $leftH \leftarrow$ HEIGHTDIAM($node.left, d$)
7: $rightH \leftarrow$ HEIGHTDIAM($node.right, d$)
8: $d \leftarrow \max(d,\ leftH + rightH + 2)$
9: **return** $1 + \max(leftH, rightH)$

12.4 (b) Worked Example

The tree with the longest path $21, 12, 5, 8, 17$ (4 edges) marked:



Lines 1–3 run once. Each node call executes lines 4, 6, 7, 8, 9; each of the 8 NULL calls (two under every leaf) executes lines 4, 5 and returns -1. The node calls finish in this order:

Node	$leftH$	$rightH$	$leftH + rightH + 2$	d after line 8	Returns
21	-1	-1	0	0	0
3	-1	-1	0	0	0
12	0	0	2	2	1
17	-1	-1	0	2	0
9	-1	-1	0	2	0
8	0	0	2	2	1
5	1	1	4	4	2

Final output: $d = 4$, which matches the expected output.

12.5 (c) Frequency Count

Line	Operation	Cost	Frequency	Count ($n = 7$)
1	$d \leftarrow 0$	1	1	1
2	call HeightDiam	1	1	1
3	return d	1	1	1
4	test $node =$ NULL	1	$2n + 1$	15
5	return -1	1	$n + 1$	8
6	left call	1	n	7
7	right call	1	n	7
8	update d	1	n	7
9	return height	1	n	7
Total $T(n)$			$7n + 5$	54

Check: $7(7) + 5 = 54$.

12.6 (d) Final Complexity

$\Theta(n)$ in the best and worst case (one visit per node). Extra space $\Theta(h)$; the worst case $\Theta(n)$ is a chain.

12.7 Twist: Why the Single Pass Beats Calling Height at Every Node

The naive method calls Height at every node and each call costs Θ(size of that subtree); on a chain this is $n + (n - 1) + \cdots + 1 = \Theta(n^2)$. Above, the height and the diameter are computed in the same pass: each of the $2n + 1$ calls does a constant amount of work, and the table shows $T(n) = 7n + 5 = \Theta(n)$ on every input, including a chain.

12.8 Program Output

Please refer to the `output.txt` file in the GitHub link: https://github.com/roh9031/EECE---7205

```
q12: diameter
4
```

13 Problem 13: Serialize and Deserialize a Binary Tree

13.1 Problem Statement

SERIALIZE turns a tree into a preorder list with the marker N for every NULL child; DESERIALIZE rebuilds the identical tree. Input: reference tree. Expected output: `5 12 21 N N 3 N N 8 17 N N 9 N N`.

13.2 Approach and Correctness

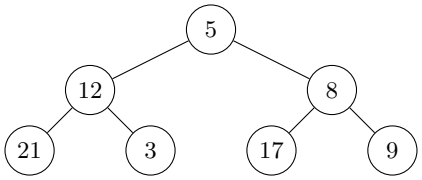
Preorder with explicit NULL markers uniquely determines the tree: reading the list in the same order, a value creates a node and the next two recursive reads build its left and right subtrees, and N ends a branch.

13.3 (a) Pseudocode

Input: tree root (serialize) / list of tokens (deserialize)
Output: list of tokens / root of the tree
Initial call: SERIALIZE(root); DESERIALIZE($list$)
Procedure SERIALIZE($root$)
1: $out \leftarrow$ empty list
2: SERHELP($root$, out)
3: **return** out
Procedure SERHELP($node$, out)
4: **if** $node =$ NULL **then**
5: append N to out; **return**
6: append $node.val$ to out
7: SERHELP($node.left$, out)
8: SERHELP($node.right$, out)
Procedure DESERIALIZE($list$)
9: $pos \leftarrow 0$
10: **return** DESHELP($list$, pos) (pos passed by reference)
Procedure DESHELP($list, pos$)
11: $tok \leftarrow list[pos]$
12: $pos \leftarrow pos + 1$
13: **if** $tok = N$ **then**
14: **return** NULL
15: $node \leftarrow$ new node(tok)
16: $node.left \leftarrow$ DESHELP($list, pos$)
17: $node.right \leftarrow$ DESHELP($list, pos$)
18: **return** $node$

13.4 (b) Worked Example

The tree:



The i-th SERHELP call appends token i, and the i-th DESHELP call reads the same token i (15 tokens $= 2n + 1$). Lines executed: each SERHELP node call runs lines 4, 6, 7, 8 and each NULL call runs lines 4, 5; each DESHELP node call runs lines 11, 12, 13, 15, 16, 17, 18 and each NULL call runs lines 11, 12, 13, 14. Lines 1–3 and 9–10 run once each.

Step	SerHelp on	Token	DesHelp action
1	5	5	create 5
2	12	12	create 12 (left of 5)
3	21	21	create 21 (left of 12)
4, 5	null, null under 21	N N	return null, null
6	3	3	create 3 (right of 12)
7, 8	null, null under 3	N N	return null, null
9	8	8	create 8 (right of 5)
10	17	17	create 17 (left of 8)
11, 12	null, null under 17	N N	return null, null
13	9	9	create 9 (right of 8)
14, 15	null, null under 9	N N	return null, null

Serialized list: `5 12 21 N N 3 N N 8 17 N N 9 N N` (matches the expected output). Deserializing it returns the root 5 with the same tree as the array.

13.5 (c) Frequency Count

Serialize (lines 1–8)

Line	Operation	Cost	Frequency	Count ($n = 7$)
1	$out \leftarrow \emptyset$	1	1	1
2	call SerHelp	1	1	1
3	return out	1	1	1
4	test $node =$ null	1	$2n + 1$	15
5	append N and return	1	$n + 1$	8
6	append $node.val$	1	n	7
7	left call	1	n	7
8	right call	1	n	7
Total $T_{ser}(n)$			$6n + 5$	47

Check: $3 + (2n + 1) + (n + 1) + 3n = 6n + 5$, and $6(7) + 5 = 47 = 1 + 1 + 1 + 15 + 8 + 7 + 7 + 7$.

Deserialize (lines 9–18; numbering continues from Serialize)

Line	Operation	Cost	Frequency	Count ($n = 7$)
9	$pos \leftarrow 0$	1	1	1
10	call DesHelp	1	1	1
11	read token	1	$2n + 1$	15
12	$pos \leftarrow pos + 1$	1	$2n + 1$	15
13	test $tok = N$	1	$2n + 1$	15
14	return null	1	$n + 1$	8
15	create node	1	n	7
16	left call	1	n	7
17	right call	1	n	7
18	return $node$	1	n	7
Total $T_{des}(n)$			$11n + 6$	83

Check: $2 + 3(15) + 8 + 4(7) = 83 = 11(7) + 6$.

13.6 (d) Final Complexity

Both procedures are $\Theta(n)$ in the best and worst case ($6n+5$ and $11n+6$ steps). Extra space $\Theta(n)$ for the token list and $\Theta(h)$ for the recursion stack (worst case $\Theta(n)$ for a chain).

13.7 Twist: Why $2n+1$ Items

The list has n values and $n+1$ markers N. Each of the n nodes has two child slots, giving $2n$ slots; every node except the root fills exactly one slot, so $n-1$ slots hold real nodes and the remaining $2n-(n-1) = n+1$ slots are NULL. Total $n+(n+1) = 2n+1$ tokens. This is also the number of recursive calls ($2n+1$), which gives the frequencies $2n+1$ and $n+1$ used in both tables above.

13.8 Program Output

Please refer to the `output.txt` file in the GitHub link: https://github.com/roh9031/EECE---7205

```
q13: serialize & deserialize
serialized: 5 12 21 N N 3 N N 8 17 N N 9 N N
deserialized root value: 5
```

14 Problem 14: Is the Tree a Valid Binary Search Tree?

14.1 Problem Statement

Return true if every node is larger than all values in its left subtree and smaller than all values in its right subtree. Input: tree P from the array, and tree S = $\{9, 5, 17, 3, 8, 12, 21\}$. Expected output: IsBST(P) = false, IsBST(S) = true.

14.2 Approach and Correctness

Pass down an allowed open range $(low, high)$. The root may take any value; a left child inherits $high$ = parent value and a right child inherits low = parent value. A node outside its range violates the BST property against some ancestor, which a check against only the two children would miss.

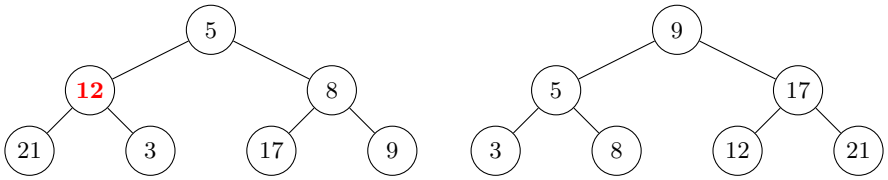
14.3 (a) Pseudocode

Input: *root*
Output: true or false
Initial call: IsBST(root)
Procedure IsBST(*root*)
1: **return** Check($root, -\infty, +\infty$)
Procedure Check($node, low, high$)
2: **if** $node$ = NULL **then**
3: **return** true
4: **if** $node.val \leq low \vee node.val \geq high$ **then**
5: **return** false
6: **return** Check($node.left, low, node.val$) $\wedge$ Check($node.right, node.val, high$)

(The **and** is short-circuited: if the left call returns false, the right call is not made.)

14.4 (b) Worked Example

Tree P (left; the violation $12 \geq 5$ is marked) and tree S (right):



Tree P: root 5 has left child 12. Line 1 runs once; each Check call then executes the lines shown.

Call	$(node, low, high)$	Lines	Check	Result
1	$(5, -\infty, +\infty)$	2, 4, 6	inside range	goes left
2	$(12, -\infty, 5)$	2, 4, 5	$12 \geq 5$	return false

The right subtree is skipped; IsBST(P) = false.

Tree S: 9; 5, 17; 3, 8, 12, 21. Each node call executes lines 2, 4, 6; each NULL call executes lines 2, 3. All 15 calls are made.

Call	$(node, low, high)$	Lines	Check	Result
1	$(9, -\infty, +\infty)$	2, 4, 6	ok	true after call 15
2	$(5, -\infty, 9)$	2, 4, 6	ok	true
3	$(3, -\infty, 5)$	2, 4, 6	ok	true
4, 5	NULL children of 3	2, 3	—	true each
6	$(8, 5, 9)$	2, 4, 6	ok	true
7, 8	NULL children of 8	2, 3	—	true each
9	$(17, 9, +\infty)$	2, 4, 6	ok	true
10	$(12, 9, 17)$	2, 4, 6	ok	true
11, 12	NULL children of 12	2, 3	—	true each
13	$(21, 17, +\infty)$	2, 4, 6	ok	true
14, 15	NULL children of 21	2, 3	—	true each

IsBST(S) = true (its inorder is $3, 5, 8, 9, 12, 17, 21$). Both results match the expected output.

14.5 (c) Frequency Count

Line	Operation	Cost	Frequency (valid tree)	Count S	Count P
1	call CHECK	1	1	1	1
2	test $node =$ NULL	1	$2n + 1$	15	2
3	return true	1	$n + 1$	8	0
4	range test	1	n	7	2
5	return false	1	0 (valid tree)	0	1
6	two calls and **and**	1	n	7	1
Total $T(n)$			$5n + 3$	38	7

Check: $5(7) + 3 = 38$ for the valid tree S; tree P stops after 2 CHECK calls.

14.6 (d) Final Complexity

Best case $\Theta(1)$ (a violation near the root, like tree P); worst case $\Theta(n)$ (a valid BST, where every node is checked). Extra space $\Theta(h)$.

14.7 Twist: Second Solution Using Inorder Traversal

A BST has a strictly increasing inorder sequence. Keep the previously visited value $prev$ (initially $-\infty$) and compare it with each visited node.

Input: $node$; global $prev = -\infty$
Output: true or false
Initial call: INCHECK(root)
Procedure INCHECK($node$)

```
1: if node = NULL then
2:    return true
3: if not INCHECK(node.left) then
4:    return false
5: if node.val ≤ prev then
6:    return false
7: prev ← node.val
8: return INCHECK(node.right)
```

Frequencies for a valid tree: line 1: $2n + 1$, line 2: $n + 1$, lines 3, 5, 7, 8: n each, lines 4, 6: 0. Total $T_{in}(n) = 7n + 2 = 51$ for $n = 7$, compared with $5n + 3 = 38$ for the range method.

On tree P the inorder method visits 21 first (the leftmost node) and then reaches $12 \leq 21$: 5 calls ($5, 12, 21$ and two NULL calls under 21), with $3 + 4 + 5 + 2 + 2 = 16$ steps, against only 7 steps for the range method. Both are $\Theta(n)$ in the worst case; the range method has the smaller constant. **Best case:** range method $\Theta(1)$ (a violation at a child of the root); inorder method $\Theta(h)$, because no comparison can be made before the leftmost node is reached.

14.8 Program Output

Please refer to the `output.txt` file in the GitHub link: https://github.com/roh9031/EECE---7205

```
q14: is valid bst
isbst(p) = false
isbst(s) = true
inorder isbst(p) = false
inorder isbst(s) = true
```

15 Problem 15: Vertical Order Traversal

15.1 Problem Statement

Give the root horizontal distance 0; a left child gets its parent's distance -1 and a right child $+1$. Print the nodes column by column from the leftmost distance to the rightmost; inside a column print top to bottom and left to right (level order). Input: reference tree. Expected output: distance -2: 21 | -1: 12 | 0: $5, 3, 17$ | $+1$: 8 | $+2$: 9.

15.2 Approach and Correctness

Level-order BFS with a queue of $(node, distance)$ pairs. Because BFS visits upper levels first and each level from left to right, appending to the column list of an ordered map ($distance \to$ list) automatically gives top-to-bottom, left-to-right order inside each column. Iterating the ordered map prints the columns from left to right.

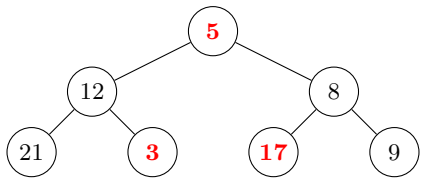
15.3 (a) Pseudocode

Input: *root*
Output: prints the nodes of each vertical column
Initial call: VERTICAL(root)
Procedure VERTICAL(*root*)

```
1:  if root = NULL then
2:    return
3:  M ← empty ordered map (distance → list)
4:  Q ← empty queue
5:  ENQUEUE(Q, (root, 0))
6:  while Q is not empty do
7:     (curr, hd) ← DEQUEUE(Q)
8:     append curr.val to M[hd]
9:     if curr.left ≠ NULL then
10:       ENQUEUE(Q, (curr.left, hd − 1))
11:    if curr.right ≠ NULL then
12:       ENQUEUE(Q, (curr.right, hd + 1))
13: end while
14: for each key d of M in increasing order do
15:    print "distance d:"
16:    for each value x in M[d] do
17:       print x
18:    end for
19: end for
```

15.4 (b) Worked Example

The tree; the horizontal distances are root 0, $12 : -1$, $8 : +1$, $21 : -2$, $3 : 0$, $17 : 0$, $9 : +2$. The three nodes in column 0 are marked:



Pass	Dequeued $(curr, hd)$	Column updated	Queue after the pass
1	$(5, 0)$	$M[0] = [5]$	$[(12, -1), (8, 1)]$
2	$(12, -1)$	$M[-1] = [12]$	$[(8, 1), (21, -2), (3, 0)]$
3	$(8, 1)$	$M[1] = [8]$	$[(21, -2), (3, 0), (17, 0), (9, 2)]$
4	$(21, -2)$	$M[-2] = [21]$	$[(3, 0), (17, 0), (9, 2)]$
5	$(3, 0)$	$M[0] = [5, 3]$	$[(17, 0), (9, 2)]$
6	$(17, 0)$	$M[0] = [5, 3, 17]$	$[(9, 2)]$
7	$(9, 2)$	$M[2] = [9]$	$[]$

Printing the map in key order: -2: 21; -1: 12; 0: $5, 3, 17$; 1: 8; 2: 9, which matches the expected output.

15.5 (c) Frequency Count

Let c be the number of columns ($c = 5$ here). Each map access is counted as one step (see (d) for its real cost).

Line	Operation	Cost	Frequency	Count ($n = 7$, $c = 5$)
1	test $root =$ NULL	1	1	1
2	return	1	0	0
3	create map	1	1	1
4	create queue	1	1	1
5	enqueue root pair	1	1	1
6	while test	1	$n + 1$	8
7	dequeue pair	1	n	7
8	append to $M[hd]$	1	n	7
9	test left child	1	n	7
10	enqueue left	1	$n - 1$ (with line 12)	3
11	test right child	1	n	7
12	enqueue right	1	(see line 10)	3
13	for-loop over columns	1	$c + 1$	6
14	print column label	1	c	5
15	for-loop over column values	1	$n + c$	12
16	print value	1	n	7
Total $T(n)$			$8n + 3c + 5$	76

Check: $8(7) + 3(5) + 5 = 76$. The grouping cost is lines 8 and 13–14 and the printing cost is lines 15–16.

15.6 (d) Final Complexity

With unit-cost map access, $T(n) = 8n + 3c + 5 = \Theta(n)$ since $c \le n$. With an ordered (balanced-tree) map, each access in line 8 costs $\Theta(\log c)$, so the time is $\Theta(n \log c)$, which is $\Theta(n \log n)$ in the worst case. The worst case is a chain made only of left children (or only of right children): every node gets its own column, so $c = n$ and the map grows to n entries. The best case is a tree with very few columns (for example $c = 1$ when every node lies in column 0, such as a single node), where the map access is $\Theta(1)$. The BFS itself does the same work for every input because every node is processed. Extra space is $\Theta(n)$ for the map and $\Theta(w)$ for the queue.

15.7 Twist: How Many Columns Can a Tree of n Nodes Have?

Distances range over $-h, \ldots, +h$, so $c \le 2h + 1$; also $c \le n$ because each column holds at least one node. A chain of n nodes made only of left children has $c = n$ columns; a perfect tree has $c = 2h + 1 = O(\log n)$. For the reference tree, $h = 2$ gives $c = 5$ (distances -2 to $+2$).

15.8 Program Output

Please refer to the `output.txt` file in the GitHub link: https://github.com/roh9031/EECE---7205

```
q15: vertical order traversal
vertical order:
distance -2: 21
distance -1: 12
distance 0: 5, 3, 17
distance 1: 8
distance 2: 9
```